

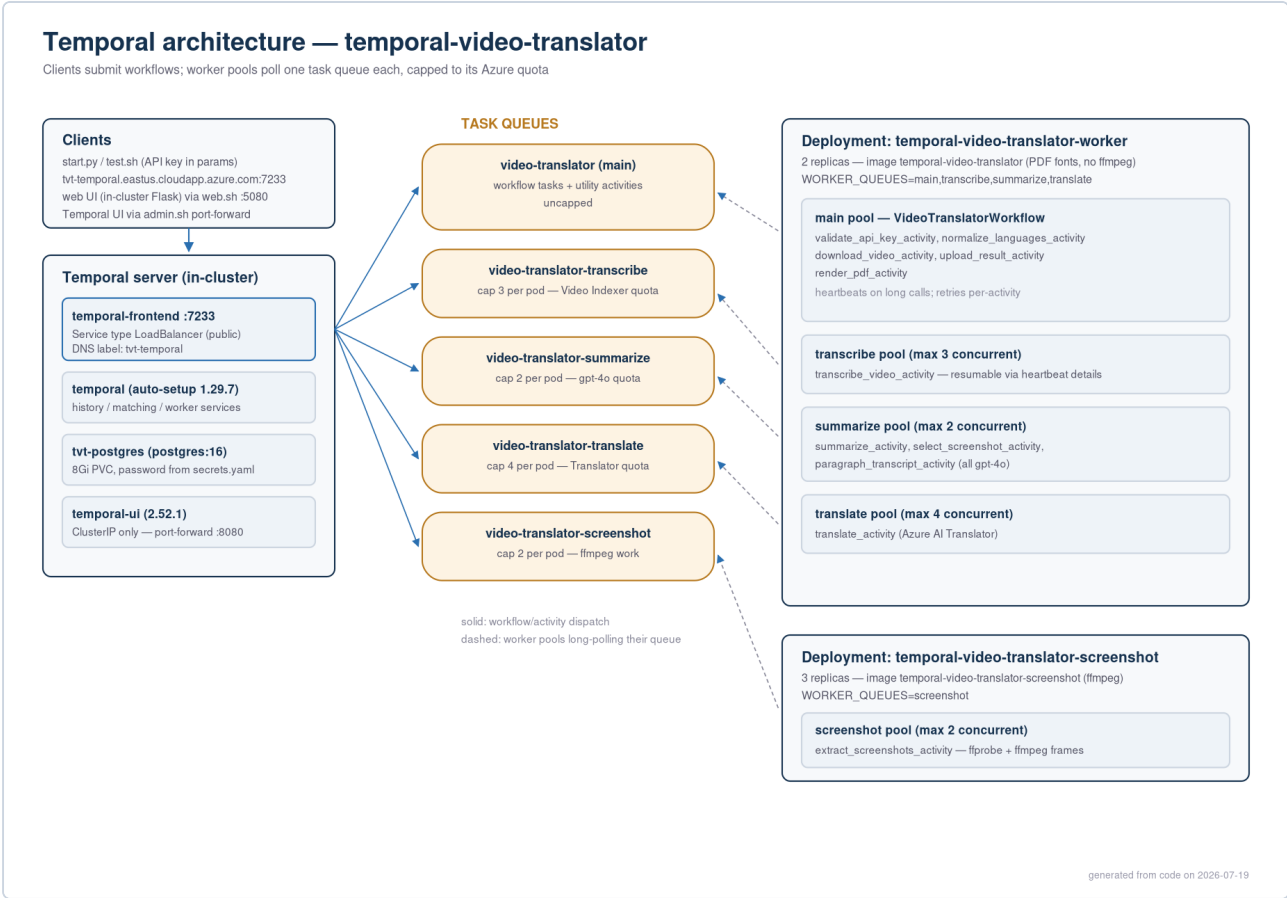
temporal-video-translator

Architecture, workflow, dependencies, and operations — v0.3.7, generated 2026-07-19

Overview

A Temporal-orchestrated pipeline on AKS that takes a video URL and a list of target languages, and produces: a speech transcript (Azure AI Video Indexer), a ~150-word summary (Azure OpenAI gpt-4o), one translation per language (Azure AI Translator), and a representative screenshot chosen by gpt-4o vision from ffmpeg-extracted frames. Results are published to public blob storage as `{workflow-id}-{video}.json`, a rendered PDF report `{workflow-id}-{video}.pdf`, and `{video}.jpg`; the workflow also returns the full document to the caller. Runs are gated by an API key validated as the workflow's first step, and can be submitted from the command line (`test.sh` / `start.py`) or the Flask web UI, which follows per-activity progress live and displays the finished results inline.

Temporal architecture



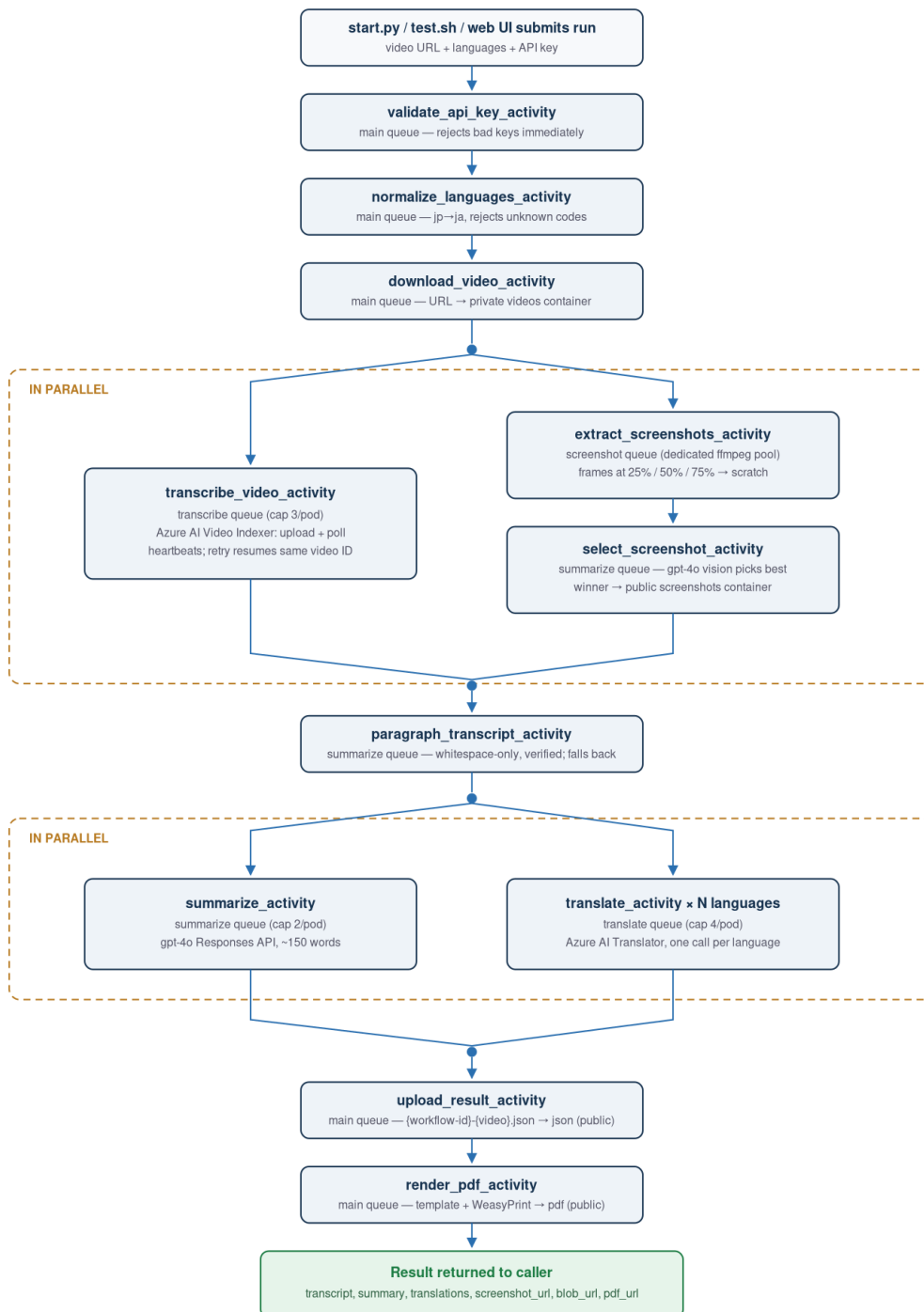
temporal.svg — queues, worker pools, and the in-cluster Temporal server

- **One workflow** (`VideoTranslatorWorkflow`) and ten activities, split across five task queues.
- **Per-resource queues with concurrency caps** protect each Azure quota independently: transcribe (3/pod, Video Indexer), summarize (2/pod, gpt-4o), translate (4/pod, Translator), screenshot (2/pod, ffmpeg CPU); the main queue (workflow + utility activities) is uncapped.
- **Worker roles**: one process hosts one Temporal worker per queue; `WORKER_QUEUES` selects the roles a container runs. The main deployment (2 replicas) runs everything except screenshots; a dedicated deployment (3 replicas, ffmpeg image) runs only the screenshot pool. Caps are per pod, so effective limits are replicas × cap.
- **Temporal server runs in-cluster** (auto-setup image + postgres + web UI), fronted by a public LoadBalancer with the DNS label `tvt-temporal.eastus.cloudapp.azure.com`.

Execution flow

VideoTranslatorWorkflow — execution flow

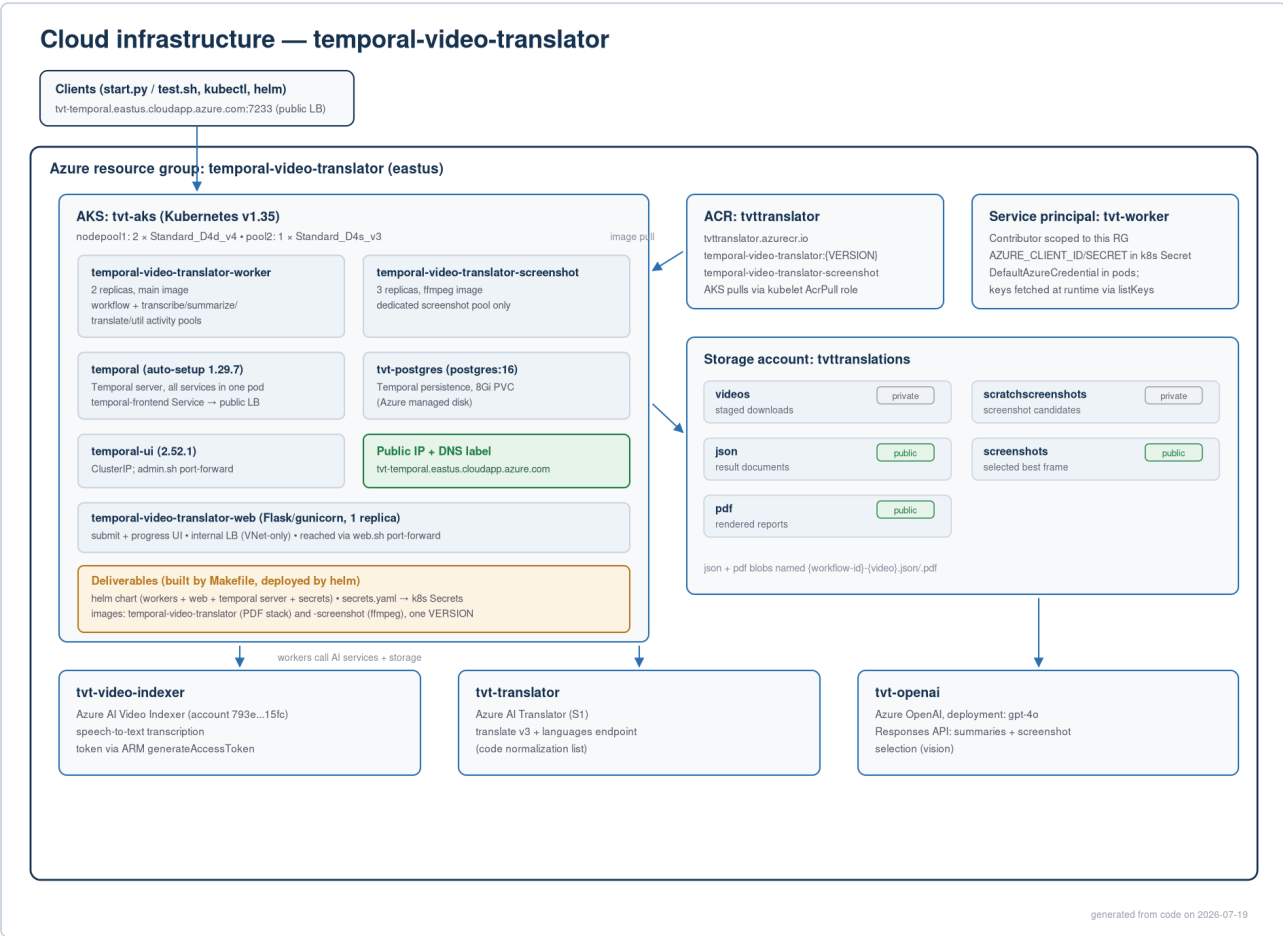
One run, top to bottom; side-by-side boxes inside dashed regions run in parallel



generated from code on 2026-07-19

flow.svg — one run; dashed regions execute in parallel

- **Fail-fast validations first:** the API key is checked, then language codes are normalized against the Translator's live language list (jp→ja, casing fixes) with unknown codes rejected — all before any paid work.
- **Parallelism:** transcription runs alongside the screenshot chain (extract → select); after the transcript lands it is broken into paragraphs (gpt-4o chooses break points by sentence number and the code inserts whitespace — verified word-for-word identical, with fallback to the raw transcript), then summarization and all translations run concurrently.
- **Resilience:** long activities heartbeat every 10s with 1-2 min heartbeat timeouts, so a dead worker is detected in minutes, not the 120-min activity timeout. Transcription stores the Video Indexer job ID in heartbeat details; a retried attempt resumes polling the same indexing job instead of re-uploading the video. Each activity has its own retry policy.



infra.svg — the temporal-video-translator resource group

- **AKS** tvt-aks : nodepool1 (2 x D4d_v4) + pool2 (1 x D4s_v3); six deployments (two worker pools, web UI, temporal server, postgres, temporal-ui).
- **Images** from ACR tvtttranslator : a main image (WeasyPrint + Noto fonts for any-script PDFs, no ffmpeg) and a screenshot image (ffmpeg, no PDF stack); one shared VERSION.
- **Storage** tvtttranslations : private staging (videos , scratchscreenshots) and public outputs (json , screenshots , pdf) — public means anonymous per-blob reads, no listing.

Dependencies

Azure service	Resource	Used for	Auth
AI Video Indexer	tvt-video-indexer	Speech-to-text transcription	ARM generateAccessToken
AI Translator	tvt-translator (S1)	Translations + language list	key via ARM listKeys
Azure OpenAI	tvt-openai / gpt-4o	Summaries + screenshot choice (Responses API)	key via ARM listKeys
Blob Storage	tvtttranslations	Staging + published artifacts	key via ARM listKeys
ACR / AKS	tvtttranslator / tvt-aks	Images / compute	kubelet AcrPull; SP in pods

Python (runtime): temporalio, requests, azure-identity, azure-storage-blob, python-dotenv, jinja2, weasyprint, flask, gunicorn. **Dev:** pytest (21 unit tests, make test). **System (in images):** ffmpeg (screenshot image); pango + Noto/DejaVu fonts (main image). Identity: a resource-group-scoped service principal (tvt-worker) feeds DefaultAzureCredential in pods; no service keys are stored anywhere — they are fetched at runtime via ARM.

Operational environment

Concern	How
Submit a run	<code>./test.sh <video-url> <lang>...</code> (or <code>start.py</code> with <code>TEMPORAL_ADDRESS</code>); API key read from <code>.env</code>
Web UI	<code>./web.sh</code> port-forward → localhost:5080 — submit form (language defaults preselected), live per-activity progress, results inline; service sits on an internal (VNet-only) LB
Endpoints	Temporal frontend: <code>tvt-temporal.eastus.cloudapp.azure.com:7233</code> (public, API-key-gated processing); UI: <code>./admin.sh</code> port-forward → localhost:8080
Build & ship	<code>make build</code> / <code>make push</code> (both images, tag from <code>VERSION</code>) / <code>make deploy</code> (helm upgrade, secrets from gitignored <code>secrets.yaml</code>)
Cluster access	<code>make creds</code> (fetch kubeconfig) and <code>make login</code> (switch context to <code>tvt-aks</code>)
Secrets	Local: <code>.env</code> . Cluster: <code>chart-root/secrets.yaml</code> → two Kubernetes Secrets (SP credentials + IDs + API key; postgres password); rotate by editing, <code>make deploy</code> , and restarting workers
Scaling	<code>worker.replicas</code> / <code>screenshotWorker.replicas</code> in <code>values.yaml</code> ; per-pod caps via <code>MAX_CONCURRENT_*</code> env — effective limit is <code>replicas × cap</code>
Local dev	<code>python worker.py</code> hosts all pools (no <code>WORKER_QUEUES</code> set); <code>make test</code> for units; <code>AZURE_TOKEN_CREDENTIALS=dev</code> skips the IMDS probe off-cloud

Known caveat: the Temporal frontend is publicly reachable and unauthenticated at the gRPC level — the API key gates processing, not visibility/termination. Restrict with `temporal.loadBalancerSourceRanges` in `values.yaml` when source IPs are known.